



Python и MPI



Клиент-серверная архитектура

Клиент-серверная архитектура - это система, которая реализует распределение как обработки, так и самих данных. Основным центральным элементом выступает сервер.

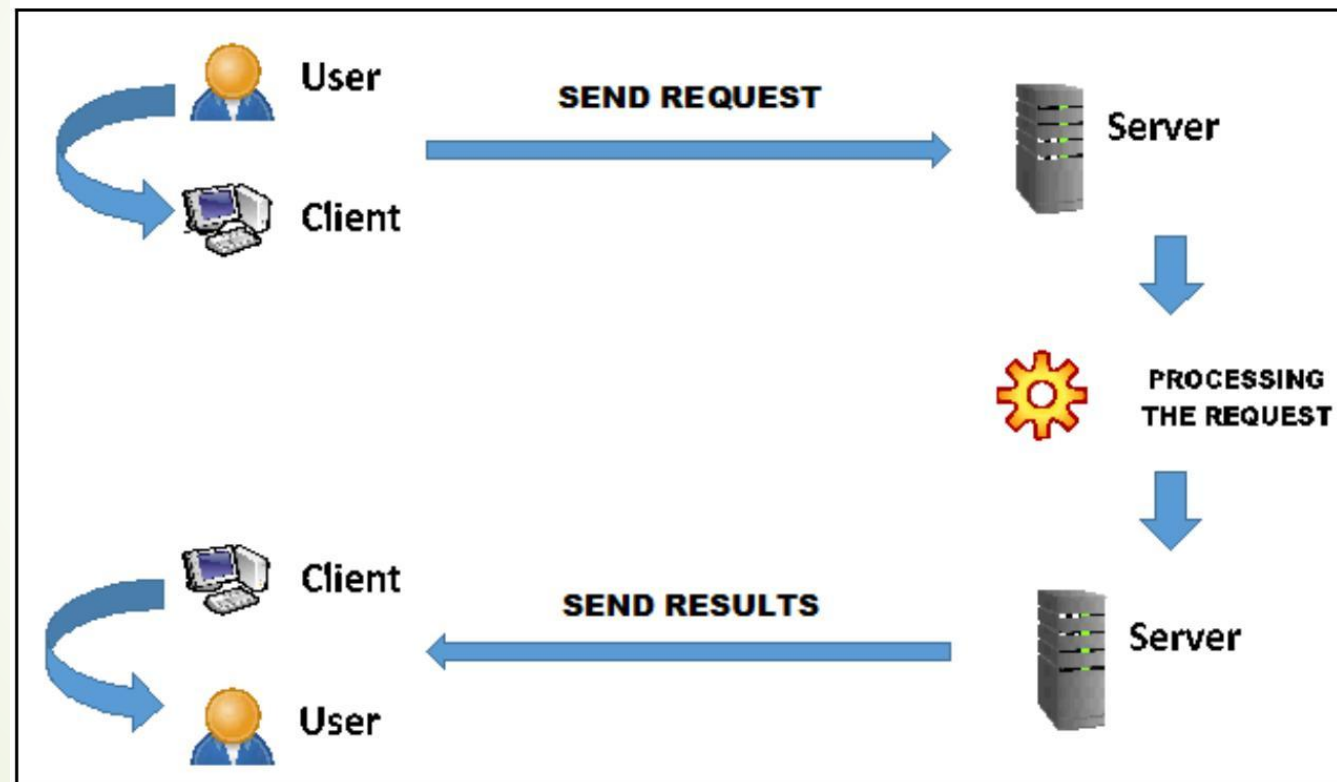
Понятие сервера можно рассматривать как с логической, так и с физической точки зрения.

С физической точки зрения - аппаратных средств - сервер выступает как машина, посвящённая исполнению серверного программного обеспечения.

С логической точки зрения сервер это программное обеспечение. Сам сервер, в качестве некоего логического процесса, предоставляет услуги прочим процессам, которые выступают в роли запрашивающей стороны или клиента.

Обычно сам сервер не отправляет свои результаты запрашивающей стороне пока эти результаты не будут затребованы его клиентом.

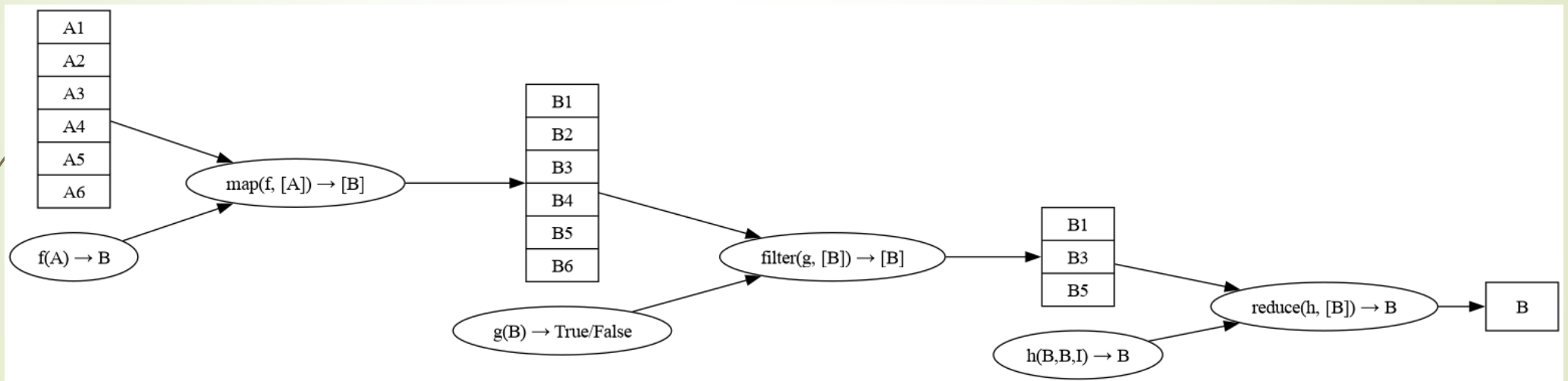
Клиент-серверная архитектура



Парадигма Map-Reduce

- MapReduce — это фреймворк для вычисления некоторых наборов распределенных задач с использованием большого количества компьютеров (называемых «нодами»), образующих кластер.
- Работа MapReduce состоит из двух шагов: Map и Reduce, названных так по аналогии с одноименными функциями высшего порядка, map и reduce.
- На Map-шаге происходит предварительная обработка входных данных. Для этого один из компьютеров (называемый главным узлом — master node) получает входные данные задачи, разделяет их на части и передает другим компьютерам (рабочим узлам — worker node) для предварительной обработки.
- На Reduce-шаге происходит свёртка предварительно обработанных данных. Главный узел получает ответы от рабочих узлов и на их основе формирует результат — решение задачи, которая изначально формулировалась.

Парадигма Map-Reduce



Запуск параллельных вычислений с использованием MPI

- MPI - программный интерфейс (API), который позволяет обмениваться сообщениями между несколькими процессами, выполняющими одну и ту же задачу. Существует несколько реализаций.
- Для работы с MPI в Windows нам понадобятся:
 - пакет `mpi4py` из `pip`
 - установить Microsoft MPI: <https://www.microsoft.com/en-us/download/details.aspx?id=57467>
- Для работы с MPI в Linux нам понадобятся:
 - пакет `mpi4py` из `pip`
 - пакет `openmpi` (системный или из `pip`)

Запуск MPI программы

➤ `mpiexec -n 4 python file.py`

где `-n 4` - число процессов при локальном запуске или `-hostfile hosts.txt` - файл с перечислением доступных узлов для запуска на вычислительном кластере.

Узлы должны быть доступны по SSH для входа от имени текущего пользователя без пароля и на них должна быть установлена совместимая версия `openmpi`.



Пример MPI

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    data = [x+42 for x in range(size - 1)]
    print("Worker", rank, "send data", data)
    for i in range(1, size):
        comm.send(data[i-1], dest=i, tag=11)
else:
    d = comm.recv(source=0, tag=11)
    print("Worker", rank, "get data", d)
```


Взаимодействие «точка-точка»

```
from mpi4py import MPI
```

```
comm=MPI.COMM_WORLD  
rank = comm.rank
```

```
print("my rank is : " , rank)
```

```
if rank==0:
```

```
    data= 10000000
```

```
    destination_process = 4
```

```
    comm.send(data,dest=destination_process)
```

```
    print ("sending data %s " %data + \
```

```
        "to process %d" %destination_process)
```

```
if rank==1:
```

```
    destination_process = 8
```

```
    data= "hello"
```

```
    comm.send(data,dest=destination_process)
```

```
    print ("sending data %s :" %data + \
```

```
        "to process %d" %destination_process)
```

```
if rank==4:
```

```
    data=comm.recv(source=0)
```

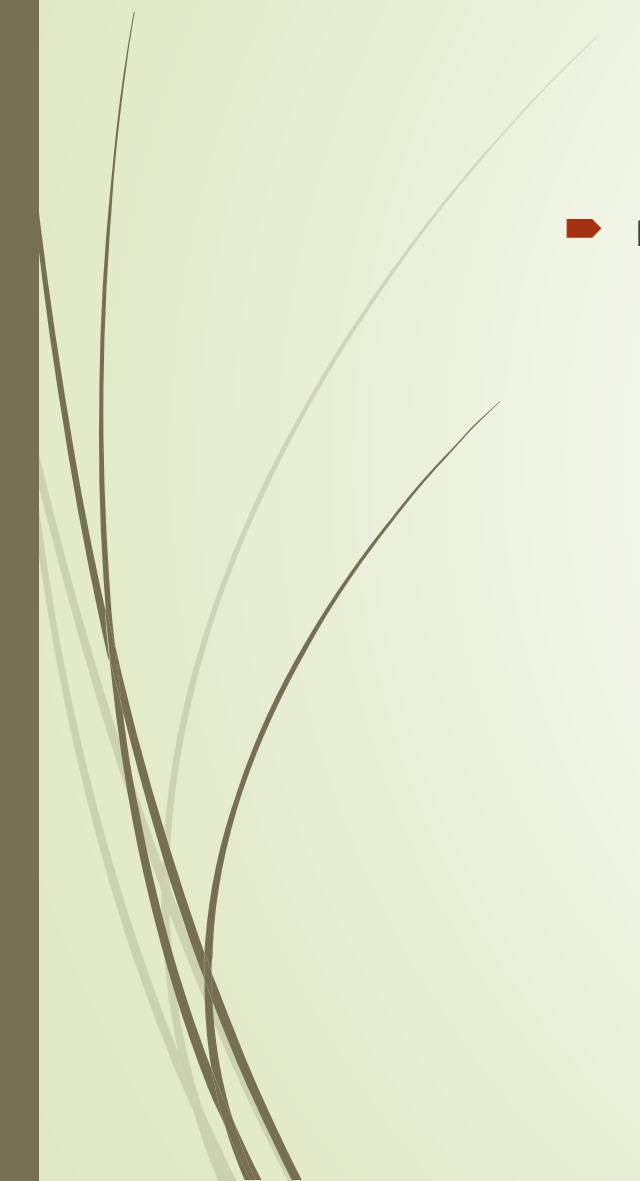
```
    data1=comm.recv(source=1)
```



Пакетное взаимодействие



`mpi2.py`

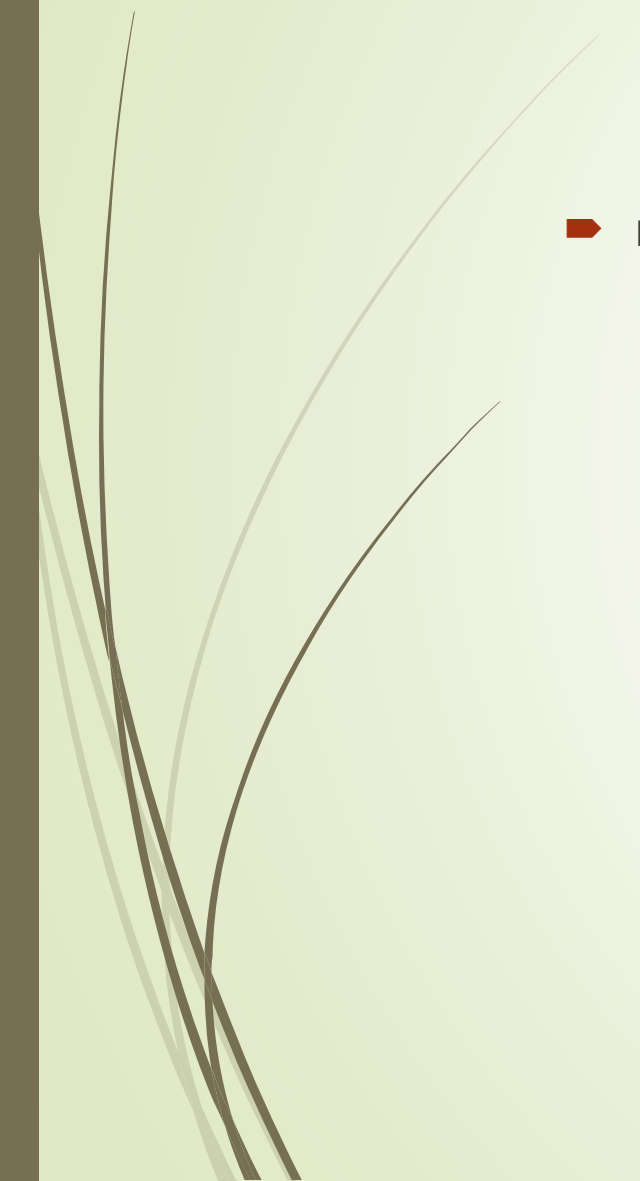




Синхронизация



mpi4.py





MPIPoolExecutor

➤ `mpiexec -n 4 python -m mpi4py.futures mpi3.py`



Чтение файла построчно

```
with open("example.txt", "r") as f:  
    for line in f:  
        print(line)
```

habr.com/ru/articles/842792/